

Задача 1. Недоступные области

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

Со времен фараоновых гробниц лабиринты строили весьма умные люди. Как правило они старались сделать так, чтобы при «посетитель» заведомо не мог попасть в самые интересные места, — скажем, в те комнаты, где хранятся главные сокровища.

В этой задаче по имеющемуся плану лабиринта нужно определить, есть ли в нем недоступные области.

После входа в лабиринт посетитель может гулять по нему как ему вздумается, не может разве что проходить сквозь стены и ходить вне лабиринта. На каждом шаге посетитель может переместиться из текущей клетки в любую соседнюю с ней по стороне пустую клетку. Ходить по диагонали нельзя.

Формат входных данных

В первой строке входного файла задано одно целое число N — размер лабиринта (лабиринт у нас квадратный, $3 \leq N \leq 1\,000$). Далее N строк содержат по N символов, описывающих сам лабиринт: стенки изображаются звездочками, а пустоты — пробелами.

Вход в лабиринт — это единственная пустая клетка в первой строке.

Формат выходных данных

В выходной файл нужно вывести одно целое число — количество недоступных областей лабиринта. Недоступные области считаются различными, если из одной нельзя пройти в другую.

Пример

input.txt	output.txt
9 *** ***** * * * ** * * * * * * * * * *** * * * * * * * ***** * * * *****	2

Комментарий

При рекурсивном обходе большого графа нужен довольно большой программный стек — явно больше, чем 1 мегабайт по умолчанию. При решении на Visual C, настоятельно рекомендуется увеличить стек, добавив следующую строчку в самом начале программы:

```
#pragma comment(linker, "/STACK:50000000")
```

Задача 2. BFS за линию

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

Дан ориентированный граф G без весов. В графе есть V вершин, занумерованных числами от 1 до N , и E дуг между некоторыми парами вершин. Требуется найти длины кратчайших путей от вершины номер 1 до всех вершин.

Формат входных данных

В первой строке входного файла находятся целые числа M и N — количество вершин и дуг в графе соответственно ($2 \leq N \leq 200\,000$, $1 \leq M \leq 200\,000$).

Следующие M строк содержат по два числа u и v ($1 \leq u, v \leq N$), означающие, что в графе есть дуга из вершины u в вершину v . По дуге можно перемещаться только в прямом направлении.

Формат выходных данных

В выходном файле требуется вывести ровно N строк. На i -й строке должно быть целое число:

- “-1”, если не существует пути от вершины 1 до вершины номер i .
- Минимальное количество рёбер в пути от вершины 1 до вершины номер i .

Пример

input.txt	output.txt
5 5	0
1 2	1
2 3	2
3 1	3
2 1	-1
3 4	

Задача 3. Списки инцидентности

Источник:	базовая
Имя входного файла:	--
Имя выходного файла:	--
Ограничение по времени:	3 секунды*
Ограничение по памяти:	128 мегабайт

Дан неориентированный граф из N вершин и M рёбер. Рёбра заданы одним списком, каждое ребро задаётся парой концевых вершин и весом. Возможны кратные рёбра и петли. Требуется за один проход по списку рёбер составить списки инцидентности для всех вершин.

От вас требуется отправить один файл `sol.c`. В нём следует подключить хедер `sol.h` следующего содержания:

```
#ifndef EDGELISTS_SOL_579846984654
#define EDGELISTS_SOL_579846984654

typedef struct {
    int from, to;
    int weight;
} Edge;

//=====
//call these functions to get the graph data:

//returns N --- the number of vertices in the graph
int getVerticesCount();
//reads the next edge from the edge list
//if there is next edge, returns 1 and stores that edge to *oEdge
//if there is no next edge, returns 0 without touching pointer
int readEdge(Edge *oEdge);

//=====
//implement these functions in your solution:

//this function is called first to initialize graph
//you are expected to read graph here and fill internal data structures
void init();

//returns number of edges indicent to vertex iVertex
int getEdgesCount(int iVertex);
//returns iIndex-th edge incident to the vertex iVertex
//it must have .from == Vertex and .to denoting the other end
Edge getIncidentEdge(int iVertex, int iIndex);

#endif
```

В `sol.c` нужно реализовать функции `init`, `getEdgesCount` и `getIncidentEdge`. Функции `getVerticesCount`, `readEdge` будут реализованы в другой единице трансляции и слинкованы вместе с вашим кодом. Для тестирования “у себя” вам желательно тоже их где-то реализовать, но отправлять в систему их не нужно.

Ограничения на размер графа: $N \leq 3 \cdot 10^5$, $M \leq 3 \cdot 10^5$. В данной задаче всё нумеруется начиная с нуля.

Гарантируется, что при тестировании функция `init` будет вызвана один раз до всех остальных вызовов. Остальные две функции могут вызываться в произвольном порядке и объёме. Гарантируется, что все вызовы корректны: `iVertex` лежит в пределах от 0 до $(N-1)$, а `iIndex` в пределах от 0 до того, что вернула ваша функция `getEdgesCount`, минус один. Всего количество вызовов `getEdgesCount` и `getIncidentEdge` не превышает 10^6 .

Обратите внимание, что каждое ребро должно входить в список инцидентности обеих своих концевых вершин. Петля должна входить дважды в список рёбер своей вершины. Порядок рёбер в каждом списке инцидентности значения не имеет.

Пример

Ниже схематично приведён порядок вызовов функций на первом тесте. Для краткости `getIncidentEdge` обозначается как `getIE`.

input.txt	output.txt
<code>5 = getVerticesCount()</code> <code>1 = readEdge() : [0, 2, 178]</code> <code>1 = readEdge() : [3, 4, 207]</code> <code>1 = readEdge() : [1, 1, 356]</code> <code>1 = readEdge() : [2, 0, 101]</code> <code>1 = readEdge() : [4, 1, 286]</code> <code>1 = readEdge() : [4, 1, 213]</code> <code>0 = readEdge()</code> <code>0 = readEdge()</code>	<code>init()</code> <code>2 = getEdgesCount(0)</code> <code>[0, 2, 178] = getIE(0, 0)</code> <code>[0, 2, 101] = getIE(0, 1)</code> <code>4 = getEdgesCount(1)</code> <code>[1, 1, 356] = getIE(1, 0)</code> <code>[1, 4, 286] = getIE(1, 3)</code> <code>[1, 1, 356] = getIE(1, 2)</code> <code>[1, 4, 213] = getIE(1, 1)</code> <code>2 = getEdgesCount(2)</code> <code>[2, 0, 178] = getIE(2, 0)</code> <code>[2, 0, 101] = getIE(2, 1)</code> <code>1 = getEdgesCount(3)</code> <code>[3, 4, 207] = getIE(3, 0)</code> <code>3 = getEdgesCount(4)</code> <code>[4, 3, 207] = getIE(4, 2)</code> <code>[4, 1, 286] = getIE(4, 0)</code> <code>[4, 1, 213] = getIE(4, 1)</code>

Задача 4. Транзитивное замыкание

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дано бинарное отношение R над множеством чисел $X = \{1, 2, 3, \dots, N\}$. Требуется найти его рефлексивно-транзитивное замыкание.

Указание: Используйте алгоритм Уоршалла (Флойда-Уоршалла).

Формат входных данных

В первой строке записано одно целое число N — размер множества ($1 \leq N \leq 500$). Далее идёт N строк по N символов в каждой, задающие отношение R . j -ый символ i -ой строки равен 1, если пара $(i, j) \in R$ (т.е. лежит в отношении R), и равен 0 в противном случае.

Формат выходных данных

Выведите N строк по N символов в каждой — рефлексивное и транзитивное замыкание отношения R , описанное в том же формате, что и исходное отношение R во входных данных.

Пример

input.txt	output.txt
5	10011
00001	11011
10010	11111
01101	10011
10000	10011
00011	

Задача 5. Побег из лабиринта

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	256 мегабайт

Для заданного лабиринта найти кратчайший путь от входа до выхода.

Формат входных данных

В первой строке входного файла находятся целые числа M и N — высота и ширина лабиринта ($1 \leq M, N \leq 100$). Каждая из следующих M строк содержит N символов, при этом символ '.' обозначает пустую клетку, символ 'X' — блоки, символ 'S' — начальную клетку, символ 'F' — конечную клетку.

Формат выходных данных

Выведите в выходной файл минимальное число шагов, за которое можно добраться от начальной клетки до конечной, каждый раз переходя на соседнюю по стороне клетку и не ступая на блоки, либо число -1 , если это невозможно.

Выходить за пределы лабиринта нельзя.

Пример

input.txt	output.txt
6 7X. .X..XF. ..XXXX. .X..... .X..X.. S...X..	11

Задача 6. Топологическая сортировка

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Имеется N лекционных тем, пронумерованных числами от 1 до N . За одну лекцию можно целиком рассказать одну тему, смешивать темы на лекции нельзя. Некоторые темы зависят от других тем. Если тема A зависит от темы B , то это означает, что нельзя рассказывать тему A , не рассказав предварительно тему B . Требуется определить, в каком порядке нужно рассказывать темы на лекции, чтобы соблюсти все зависимости.

Если есть несколько подходящих порядков, нужно выбрать лексикографически наименьший из них. То есть номер первой рассказываемой темы должен быть минимально возможный, среди таких порядков нужно выбрать такой, у которого номер темы на второй лекции минимально возможный, далее нужно минимизировать номер темы на третьей лекции, и так далее.

Указание: Используйте алгоритм Кана, но с доработкой. Чтобы ускорить алгоритм Кана, для каждой темы храните и поддерживайте количество ещё не рассказанных тем, от которых она зависит.

Формат входных данных

В первой строке записано два целых числа: N — количество тем и M — количество зависимостей ($1 \leq N \leq 5\,000$, $1 \leq M \leq 100\,000$).

В оставшихся M строках описаны зависимости. Каждая зависимость описывается двумя целыми числами B и A , что означает, что тема A зависит от темы B ($1 \leq A \neq B \leq N$).

Формат выходных данных

Выведите искомый порядок в единственную строку выходного файла: N целых чисел, обозначающих номера тем. Если в темах имеется циклическая зависимость и искомого порядка не существует, выведите слова “bad course” вместо порядка.

Пример

input.txt	output.txt
5 7 1 5 1 3 5 2 5 4 3 4 1 2 4 2	1 3 5 4 2
3 3 1 2 2 3 3 2	bad course

Задача 7. Транзитивное замыкание+

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дано бинарное отношение R над множеством чисел $X = \{1, 2, 3, \dots, N\}$. Требуется найти его рефлексивно-транзитивное замыкание.

Формат входных данных

В первой строке записано одно целое число N — размер множества ($1 \leq N \leq 2\,000$). Далее идёт N строк по N символов в каждой, задающие отношение R . j -ый символ i -ой строки равен 1, если пара $(i, j) \in R$ (т.е. лежит в отношении R), и равен 0 в противном случае.

Формат выходных данных

Выведите N строк по N символов в каждой — рефлексивное и транзитивное замыкание отношения R , описанное в том же формате, что и исходное отношение R во входных данных.

Пример

input.txt	output.txt
5	10011
00001	11011
10010	11111
01101	10011
10000	10011
00011	

Задача 8. Топологическая сортировка+

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда*
Ограничение по памяти:	разумное

Имеется N лекционных тем, пронумерованных числами от 1 до N . За одну лекцию можно целиком рассказать одну тему, смешивать темы на лекции нельзя. Некоторые темы зависят от других тем. Если тема A зависит от темы B , то это означает, что нельзя рассказывать тему A , не рассказав предварительно тему B . Требуется определить, в каком порядке нужно рассказывать темы на лекции, чтобы соблюсти все зависимости.

Если есть несколько подходящих порядков, нужно выбрать лексикографически наименьший из них. То есть номер первой рассказываемой темы должен быть минимально возможный, среди таких порядков нужно выбрать такой, у которого номер темы на второй лекции минимально возможный, далее нужно минимизировать номер темы на третьей лекции, и так далее.

Указание: Зависимости нужно хранить не в матрице, а в виде массива списков (связных списков или растущих массивов). Кроме этого, нужно ускорить алгоритм Кана до асимптотики $O(M + N \log N)$

Формат входных данных

В первой строке записано два целых числа: N — количество тем и M — количество зависимостей ($1 \leq N \leq 100\,000$, $1 \leq M \leq 200\,000$).

В оставшихся M строках описаны зависимости. Каждая зависимость описывается двумя целыми числами B и A , что означает, что тема A зависит от темы B ($1 \leq A \neq B \leq N$).

Формат выходных данных

Выведите искомый порядок в единственную строку выходного файла: N целых чисел, обозначающих номера тем. Если в темах имеется циклическая зависимость и искомого порядка не существует, выведите слова “bad course” вместо порядка.

Пример

input.txt	output.txt
5 7 1 5 1 3 5 2 5 4 3 4 1 2 4 2	1 3 5 4 2
3 3 1 2 2 3 3 2	bad course

Задача 9. Vim

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	5 секунд
Ограничение по памяти:	256 мегабайт

Описанные в данной задаче команды могут отличаться от команд настоящего редактора Vim. Пожалуйста, внимательно читайте условие.

В редакторе Vim текст отображается построчно, при этом строки текста могут быть разной длины. Текст состоит из строчных букв латинского алфавита и пробелов.

Для навигации по тексту используется курсор. В редакторе Vim курсор находится не между символами, а на символе. Его положение в тексте задаётся парой координат: номером строки и позицией в строке. Положение курсора всегда соответствует реальному символу в тексте: номер строки не может быть меньше 1 или больше количества строк в тексте, а позиция не может быть меньше 1 или больше длины текущей строки.

Положение в тексте A считается расположенным раньше положения B , если:

- номер строки A меньше номера строки B ;
- оба положения имеют одинаковый номер строки, но позиция A меньше позиции B .

Соответственно положение A считается расположенным позже положения B , если B расположено раньше A .

Словом в тексте называется последовательность букв латинского алфавита. Слова разделяются одним или несколькими пробелами и переводами строки. Началом слова называется первый символ слова, а концом — последний символ слова.

Vim имеет несколько различных режимов работы, один из которых — режим команд. В режиме команд можно использовать следующие клавиши для навигации по тексту:

- Клавиша 'l' сдвигает курсор на одну позицию вправо. Если курсор стоял на последнем символе строки, то он никуда не перемещается.
- Клавиша 'h' сдвигает курсор на одну позицию влево. Если курсор стоял на первом символе строки, то он никуда не перемещается.
- Клавиша 'w' перемещает курсор в начало следующего слова в тексте: наименьшее положение, соответствующее началу какого-либо слова, и которое идёт позже текущего положения курсора. Если такого начала слова нет, то курсор никуда не перемещается.
- Клавиша 'b' перемещает курсор в начало текущего слова: наибольшее положение, соответствующее началу какого-либо слова и которое идёт раньше текущего положения курсора. Если такого начала слова нет, то курсор никуда не перемещается.
- Клавиша '\$' перемещает курсор на последнюю позицию текущей строки.
- Клавиша '0' (символ нуля) перемещает курсор на первую позицию текущей строки.
- Клавиша 'j' перемещает курсор на строчку вниз. Если строка последняя, то курсор не перемещается.

- Клавиша 'k' перемещает курсор на строчку вверх. Если строка первая, то курсор не перемещается.

При перемещении вверх и вниз курсор запоминает, с какой позиции было начато перемещение между строками — назовём эту позицию стартовой. Если курсор перемещается на строку, длина которой больше или равна номеру стартовой позиции, то в новой строке положение курсора будет иметь стартовую позицию. Если курсор перемещается на строку, длина которой меньше стартовой позиции, то курсор перемещается на последнюю позицию новой строки. При дальнейшем нажатии на клавиши 'k' или 'j' курсор так же выбирает позицию в новой строке исходя не из текущей позиции, а из стартовой позиции, с которой началось перемещение между строками. Память о стартовой позиции стирается после нажатия на клавишу, отличную от 'k' или 'j'.

Рассмотрим работу клавиши 'k' на примере. Пусть есть три строки длиной 16, 6 и 11, и курсор находится в третьей строке в позиции 9. Если первый раз нажать на «вверх», то курсор переместится из положения (3, 9) в положение (2, 6). Если второй раз нажать на «вверх», то курсор переместится из положения (2, 6) в положение (1, 9), т.к. движение между строками началось с позиции 9. Если же между первым и вторым нажатием сделать перемещение курсора влево, то перемещения курсора будут: (3, 9) → (2, 6) → (2, 5) → (1, 5).

Вы — эффективный программист, потому в качестве основного редактора выбрали Vim. Поскольку вы эффективны, то хотите все действия выполнять за минимальный объём приложенных усилий. Перед вами, как перед программистом, часто стоит задача переместить курсор из одного положения в другое и, конечно, вы хотите это сделать за минимальное количество нажатий на клавиши. Помогите себе: напишите программу, которая по заданному тексту и заданным начальному и конечному положениям курсора найдёт кратчайшую последовательность нажатий на клавиши, перемещающую курсор из начального положения в конечное.

Формат входных данных

В первой строке содержится единственное целое число N ($N > 1$) — количество строк в тексте.

Следующие N строк содержат текст. Для удобства все символы пробела в тексте заменены на символ точки (ASCII 46). Текст состоит из строчных букв латинского алфавита и пробелов (заменённых на точки). Каждая строка текста содержит как минимум один символ, однако в ней могут отсутствовать буквы. Длина текста (суммарная длина всех строк) не превосходит 10^5 .

Следующая строка содержит число Q ($1 \leq Q \leq 10^3$) — количество запросов на перемещение курсора.

Каждый запрос описан на отдельной строке. Запрос состоит из 4 чисел, описывающих начальное и конечное положение курсора: *rows*, *cols*, *rowf* и *colf* — соответственно номер строки и позиция в этой строке начального положения курсора и номер строки и позиция в этой строке конечного положения курсора. Нумерация строк и позиций начинается с 1. Гарантируется, что данные координаты корректны и соответствуют реальным позициям в тексте. Начальное и конечное положения курсора не совпадают.

Произведение длины текста на количество запросов не превышает 10^7 .

Формат выходных данных

Ответом для каждого запроса является кратчайшая последовательность нажатий клавиш, перемещающая курсор из начального положения в конечное. Для каждого запроса выведите ответ на него на отдельной строке. Если существует несколько кратчайших последовательностей, выведите любую из них.

Пример

input.txt	output.txt
3 abc.def q qewrewqr 4 3 7 3 5 1 2 1 5 1 2 1 6 1 1 3 5	hh w \$h wj